

Compression d'image par DCT et stockage CSR en C++

Romain BEN, Karim ZRIG | MAM4 Polytech Nice Sophia, projet C++ | septembre 2026

Code source : <https://github.com/BnRomain/jpeg-compression>, dossier `cpp/`

1. Objectif

En MAM3, nous avons programmé en Python une compression d'image inspirée de JPEG, avec un module de calcul (`jpeg_compression.py`) et une application Streamlit : DCT sur des blocs 8×8 , quantification, suppression des hautes fréquences et stockage creux au format CSR avec `scipy`. Ce projet réécrit l'ensemble en C++20, sans bibliothèque de calcul, en s'appuyant sur les notions du cours. Le programme en ligne de commande `jpeg_csr` lit une image PNG ou JPEG, la compresse, écrit les trois matrices CSR dans un fichier binaire `.csr`, reconstruit l'image et affiche les indicateurs de l'application Python (taux de conservation, erreur, tailles mémoire). Les analyses du rapport MAM3 sont disponibles par options : facteur de qualité α , matrices Q alternatives, troncature carrée (version Python) ou triangulaire (sujet), bruit poivre et sel.

2. Algorithme

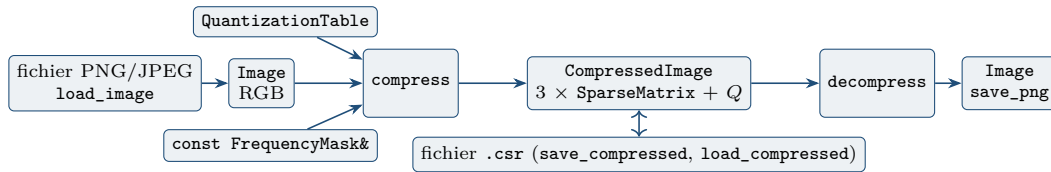
Pour chaque canal R, G, B et chaque bloc M d'intensités centrées dans $[-128, 127]$:

$$D = P M P^T, \quad P_{k,i} = \frac{C_k}{2} \cos \frac{(2i+1)k\pi}{16}, \quad \hat{D}_{k,l} = \text{trunc}\left(\frac{D_{k,l}}{\alpha Q_{k,l}}\right),$$

avec $C_0 = 1/\sqrt{2}$ et $C_k = 1$ sinon. $\hat{D}_{k,l}$ est ensuite annulé si $|\hat{D}_{k,l}| < S$ (seuil) ou si le masque le rejette : le masque carré garde $k, l < F$, le masque triangulaire $k + l < F$. Les coefficients d'un canal forment une matrice presque vide, stockée en CSR (valeurs `int16`, indices de colonnes et pointeurs de lignes `int32`, comme `scipy`). La décompression calcule $\tilde{M} = P^T(\hat{D} \odot \alpha Q)P + 128$, borné à $[0, 255]$.

3. Architecture du code

Le code suit la compilation séparée du cours : un en-tête commenté par module dans `include/`, les définitions dans `src/`, un Makefile (`-std=c++20 -Wall -Wextra -pedantic`) et les tests dans `tests/`.



Module	Rôle	Notions du cours
Image	image RGB, rognage aux multiples de 8	invariant, <code>std::vector</code> , accès <code>const</code> ou non
Matrix8, Dct	bloc 8×8 , matrice P calculée une seule fois	<code>std::array</code> , surcharge d'opérateurs
QuantizationTable	Q standard, uniforme, filtrantes, facteur α	invariant, constructeur <code>explicit</code> , exceptions
FrequencyMask	interface de <code>SquareMask</code> et <code>TriangleMask</code>	classe abstraite, <code>virtual</code> , <code>override</code>
SparseMatrix	format CSR écrit à la main	invariant vérifié, déplacement <code>std::move</code>
CompressedImage	3 matrices CSR et Q , fichier <code>.csr</code>	composition, flux binaires
image_io	lecture et écriture d'images via <code>stb</code>	RAII, copie interdite (= <code>delete</code>)
codec, metrics, noise	compression, erreurs, bruit	<code>const T&</code> en lecture, <code>T&</code> en modification

Invariants. Chaque classe établit son invariant dans son constructeur et lance `std::invalid_argument` sinon ; ses données sont privées. L'invariant de `QuantizationTable` (diviseurs ≥ 1) garantit $|\hat{D}_{k,l}| \leq 1024$ et justifie le type `int16`. Celui de `SparseMatrix` est revérifié à la relecture d'un fichier `.csr`, ce qui rejette un fichier corrompu.

Ressources. Les classes composent `std::vector` ou `std::array` : aucune n'écrit de destructeur ni d'opération de copie (règle de zéro). La seule ressource manuelle, le tableau alloué par `stbi_load`, est confiée à la classe interne `StbPixels` (RAII, copie interdite) ; la bibliothèque `stb` (domaine public) reste confinée dans `image_io.cpp`.

Polymorphisme. `compress` reçoit un `const FrequencyMask&` et appelle `keeps(k, 1)` sans connaître la stratégie. `main` construit sur la pile le masque demandé et le passe par référence de base : le choix se fait à l'exécution, sans `new` ni `delete`, et ajouter un masque ne modifie pas `compress`.

4. Validation

`make test` exécute 17 fonctions de test à base d'`assert` : les tests `pytest` de la version Python (orthogonalité de P , rognage, suppression des hautes fréquences, bornes, conversion CSR), les invariants de chaque classe, l'aller-retour par fichier `.csr` et l'analyse de la ligne de commande. Une intégration continue GitHub les relance sous `AddressSanitizer` et `UBSan`. Le script `compare_with_python.py` compresse la même image avec les deux versions : 786 407 coefficients sur 786 432 sont identiques, avec autant de non nuls (45 566). Les 25 autres diffèrent de 1 : en Python l'image passe par $/255$ puis $\times 255$, D/Q vaut par exemple 23,999 999 999 999 996 au lieu de 24 et la troncature donne 23.

5. Résultats

Image `astronaut.png` (512×512 , 773 Ko ; 6 144 Ko en `float64`), seuil $S = 2$, Q standard et masque carré $F = 6$ sauf mention contraire.

Réglage	Conservation	Erreur L^2	PSNR	Taille CSR	Gain dense/CSR
$\alpha = 1$ (réglages Python)	5,79 %	5,43 %	30,5 dB	273 Ko	22,5
$\alpha = 5$	2,15 %	11,29 %	24,1 dB	105 Ko	58,5
$\alpha = 20$	0,64 %	32,95 %	14,8 dB	35 Ko	174,0
Q uniforme (100)	1,65 %	13,85 %	22,4 dB	82 Ko	75,1
Q basses fréquences, sans masque	8,76 %	6,52 %	28,9 dB	409 Ko	15,0
Q hautes fréquences, sans masque	25,45 %	17,49 %	20,3 dB	1 179 Ko	5,2
Masque triangulaire $F = 2$	2,91 %	11,41 %	24,0 dB	140 Ko	43,8



FIGURE 1 – À gauche : originale, $\alpha = 1$, 5 et 20 (zoom). À droite : image pure, bruitée à 5 % (poivre et sel), reconstruite avec un masque triangulaire $F = 4$.

Comme en MAM3, α règle le compromis entre taux et qualité : à $\alpha = 20$, presque tous les coefficients AC disparaissent et l'image devient une mosaïque de blocs. La matrice uniforme garde 1,65 % des coefficients pour 13,9 % d'erreur, quand la matrice psychovisuelle avec $\alpha = 5$ en garde 2,15 % pour 11,3 % ; la matrice qui privilégie les hautes fréquences conserve un quart des coefficients pour une image moins fidèle. Avec 5 % de bruit poivre et sel, l'erreur par rapport à l'image pure passe de 24,0 % (image bruitée) à 11,3 % après compression : la troncature agit comme un filtre passe-bas, au prix d'un flou. **Performances** : compression et décompression prennent 50 ms en C++ contre 543 ms en Python sur la même machine (meilleur de trois essais), soit un facteur 11, grâce aux boucles compilées en -O2 au lieu d'appels numpy bloc par bloc.

6. Bilan

Le portage reproduit la version Python au coefficient près, aux arrondis flottants près, et l'exécute onze fois plus vite. Les notions du cours ont guidé la structure : les invariants rendent explicites des hypothèses implicites en Python (dimensions multiples de 8, type `int16` des coefficients, cohérence d'un fichier CSR) et la règle de zéro supprime toute gestion mémoire manuelle. Pistes : parcours zigzag et codage entropique (RLE, Huffman) pour produire un vrai fichier JPEG, passage en YCbCr, et stockage de plusieurs masques polymorphes avec `std::unique_ptr`.